

Selection, Not Capacity

A measured decomposition of retrieval failure in conversational memory, and what survived eleven negative results

Idris Applied AI Research — independent, non-profit

Repository: contextDecayWindow · Licence: CC BY 4.0

Draft — PAPER-001

Executive summary

What was tried. Eleven pre-registered attempts to give a language model a working memory of a long conversation: store each exchange, then rebuild a small context every turn. Ten numbered studies plus one retrieval bakeoff, across summarizers, promotion filters, a topic layer, a knowledge graph, a query router, and approximate search.

What survived. None of those mechanisms cleared its own success criterion. What is left is store everything verbatim, keep the recent turns, retrieve by similarity, and pick a set that covers different topics — with no generative model calls anywhere in the memory path.

Three findings.

1. **The context budget was never the limit.** A perfect picker reaches the target using a sixth of the available space. The deployed system spent all of it and delivered a third as much.
2. **For “list everything about X” questions, the most similar records were the least useful.** The four closest matches to the question carried none of its answers. For ordinary lookup questions the same ranking was near-perfect. That split rests on one enumeration question, and testing it elsewhere is the open problem this work leaves.
3. **Fix order matters, and the intuitive order is wrong.** The deployed shortlist contained no record at all from one of the four topics, so no selection rule of any kind could cover all four from it. Improving the rule before widening the shortlist cannot work, and measured once, it made things slightly worse than the baseline.

Three operational instructions.

- **Do not prune the candidate pool to control cost.** Dropping the least similar records is the one operation measured here to break retrieval — it cost an entire topic even though most of the best records survived the cut.
- **Check whether your memory layer needs model calls at all.** Every component removed here was one that required them.
- **Know your latency horizon before designing for scale.** Storage is free; retrieval time is not.

Cost and risk. Disk is trivial — about 48 MB at ten thousand turns. Retrieval time binds well before that: 190 ms at a thousand stored records, with most of it in one stage whose share is still growing. On this hardware the design is comfortable to a few thousand records and unusable in an interactive loop somewhere before ten thousand.

What this does not establish.

- One conversation corpus, one model, one machine, one seed. No error bars anywhere, and no comparison against any published system.
- The final configuration was **never run live**. Every count here measures whether a fact was *present in the window*, not whether the model answered correctly.
- The breadth findings rest on a single enumeration question.

Two images carry the argument between them. Figure 1 is the budget claim: the target is affordable and deployed selection spent everything without reaching it. Figure 3 is the ordering claim, and it is the one drawn at the resolution the committed record allows — 16 of 119 points, with its key readings as annotations rather than plotted data.

Abstract

Eleven pre-registered attempts to build a memory layer for long conversations produced one architecture worth keeping and a measurable account of why the rest failed. This is a single-program experience report: one corpus, one model, one seed, no external calibration, and no error bars.

The failures share a cause that can be measured. Write-time selection failed in five studies because it cannot anticipate a later query. Moving selection to query time failed in a registered bakeoff, which reached fewer target facts than write-time formation had. Graph construction, query-type routing, approximate search, query segmentation, and attention-derived term selection each failed their own gates. Underneath all of them sits a candidate ordering that, for this corpus’s one enumeration probe, is anti-correlated at the top: the four highest-similarity episodes carry none of its target facts and the last needed item does not appear until rank 87 of 119. The same ordering places every needed item inside rank 2 on all eight lookup probes, so this is one probe behaving unlike eight rather than an established property of query types.

Separating the failure gives three constraints that bind in a forced order, and the order is the paper’s operational result. **The candidate pool binds first, structurally:** the art domain has no representative anywhere in the deployed 34-episode pool, so no selection rule of any kind reaches four domains from it — this follows from the pool’s contents, not from any measured comparison. With the selector held fixed, widening the pool to 119 episodes moves 5 of 17 items across 2 domains to 12 across 4. **The objective binds second, and only after that;** run on the deployed pool the shipped objective scores 5 of 17 against the 6 of 17 baseline it replaces, one count from one run, illustrating the ordering rather than establishing it. **A similarity floor binds last:** the final missing episode needs a query cosine of 0.225 and has 0.056, which no reweighting closes.

Capacity was never the constraint. An exact optimum on the same store makes 14 of 17 items available in 5,058 of 32,000 characters; deployed selection made 6 available while spending 31,946. These are availability counts measured offline against a planted answer key, and the resulting configuration was never run live. What remains after every one of those removals is an append-only store, a recency window, similarity retrieval, and a coverage objective, with no generative model calls in the memory path — a design that is reproducible and free of generated intermediate text because the removed components were the ones that produced it.

Reading this paper

Terms. The program’s vocabulary, defined once.

Term	Meaning here
Episode	One stored user message and its assistant reply, kept verbatim. The unit of storage, retrieval, and cost throughout
Store	Every episode of one conversation. The §5 store holds 119 episodes eligible for its final probe; §6.4’s holds 1,000
Probe	A scripted question asked at a known turn, with a locked answer key
Targeted probe	Asks for one specific fact — “who led the survey?” Eight of them
Breadth probe	Asks the model to enumerate everything it knows across topics. The program has exactly one , at turn 120, worth 17 items
Domain	One of the conversation’s four subject areas: civil engineering, art history, monetary policy, marine biology. The breadth probe’s 17 items span all four
Cosine	Similarity between two embedding vectors, 0 to 1. Used two ways below: a cosine value (0.056) and a cosine rank , an episode’s position when the store is sorted by that value (rank 112 of 119)
Availability	Whether an item’s text was present in the delivered context. Not whether the model answered correctly
Budget	A hard ceiling of 32,000 characters on the delivered context, charged at exact serialized cost including tags and separators
Candidate pool	The episodes a selector is allowed to consider, after any pre-filter
Selector / objective	The rule choosing which candidates to deliver. The deployed one ranks each episode independently; a <i>set-level</i> objective scores an episode by what it adds to those already chosen

Work identifiers. Studies are numbered 001–010. Later work carries prefixes: **AR** achievability, **DR** diagnostic repair, **DX** diagnostic, **E** mechanism experiment, **CC** component closeout. The five that matter here:

ID	What it did
AR-001	Computed the cheapest set of episodes satisfying the breadth probe

ID	What it did
E005	Swept three set-level selectors over 146 configurations
DR-002	Froze one configuration and varied only the candidate pool
DX-001	Asked why one known-optimum episode is never selected
DX-002	Asked whether a 1,000-turn run's context was still growing

Route. For the argument: §5, then §6. For the methodology lessons: §7.2 and §7.3, which are the most transferable content here. §4 is background.

1. Introduction

A long conversation forces a trade. Keep the transcript and the model slows down and loses the middle. Summarize it and the details are gone. This program spent eleven pre-registered efforts on a third option: store every exchange verbatim and rebuild a small, relevant context each turn.

Most of those efforts failed. This paper is about what the failures had in common, which turned out to be measurable, and about what was left after the failed parts were removed, which turned out to be small.

1.1 The category, declared

This is a case study of one program, not a general-claims paper, and the distinction decides which sentences are permitted. The program was never externally calibrated: Study 003 retired external baselines in favour of self-comparison and nothing restored them. Every number comes from one corpus, one rubric locked since Study 002, one local model at one quantization, one machine, one seed. Every comparison is a single run, so there is no variance estimate anywhere and no significance test that would mean anything.

Findings below are therefore stated as *on this corpus*, and where a result would be worth testing elsewhere the paper names the experiment rather than implying its outcome. §8 states the limits once, in full, rather than re-stating them at every claim.

1.2 Contributions

1. **A decomposition of where retrieval failure lives** (§5): a candidate pool, a selection objective, and a similarity floor, each separately bounded. **They are not independent — they bind in a forced order**, and applying the second fix without the first makes the shipped configuration worse than the baseline it replaces (§5.6).
2. **The measurement that makes the decomposition possible**: a per-fact known optimum computed on the same store under exact serialized-cost accounting (§5.1). It needs an answer key and exact cost accounting, which is why it is unusual rather than difficult.
3. **A subtraction result** (§6): what eleven efforts removed, and why the properties that make what remains deployable follow from the negative results.
4. **A correction record** (§7), including one instance where a diagnostic written to catch a specific failure class nearly committed that exact failure.

1.3 What this paper does not claim

No comparison against HippoRAG, Mem0, Zep, or Letta; none were run. No general claim that similarity retrieval fails — §5.5 measures the opposite on eight of nine probes. No novelty for maximal marginal relevance, facility location, or submodular selection, which are established methods; what is offered is the decomposition and the measurement, not the selector. And no claim that 12 of 17 is good: it sits below the program's registered bar of 14, and below the 15 a known optimum reaches on the same store for a sixth of the cost.

2. Related work

Studies 001–010 were designed and run before this program's first literature scan and are committed with SHAs that show it. We state that once, because it explains why early designs rediscovered known ideas, and press it no further: arriving independently at a worse version of a published method is not a contribution.

Entity-centric indexing. HippoRAG builds a knowledge graph over extracted entities and retrieves by traversal; the authors' own follow-up identifies entity-centricity as a limitation. This program's hardest repeated failure is consistent with that. Six target facts sit in spans where the entity extractor finds zero

entities, so an entity-gated index has no path to them — which is why entity extraction was ruled out as a primary index here.

Structure over retrieved units. GraphRAG, SGMem, and CodaRAG impose explicit structure on retrieved material. This program built the corresponding mechanism, an associative graph over observed co-activation, and no configuration cleared its advancement gate (§4).

Deployed systems and benchmarks. Letta, Mem0, and Zep ship in this space; LoCoMo and Long-MemEval are the calibration this program lacks. Both groups were adopted in principle and never run, so this paper makes no comparison to either.

The placement is narrow: every system above consumes a candidate set produced upstream by similarity ranking, and this paper measures that set rather than proposing another structure over it.

3. Method

Pre-registration and binding gates. Each study’s design was committed before implementation, and that commit’s SHA is the integrity anchor. Offline gates run before inference and bind: Study 008 stopped at its pre-run gates when replay proved no registered fill cap between 1 and 50 could pass the breadth and targeted gates jointly, preventing four invalid 121-turn runs. Amendments never edit a locked registration; they are standalone files recording whether they preceded the result they affect, and the bakeoff carries twelve.

Exact serialized cost. Every character budget charges the complete serialized block — per-episode tags, metadata, and separators included, not just source text. This was not true before DR-001 (§7.2), and correcting it moved published numbers by up to 68%. Every figure here uses the corrected accounting.

Determinism and leakage control. Fixed seed, one slot, speculative decoding disabled, and a required byte-identical seeded-prefix rerun. Mechanism code — retrieval, formation, ranking, gating — may not read the answer key; measurement may. The boundary is enforced by grep, import-graph checks, and a planted test violation that must be caught. Controls run from checked-out prior code in a separate worktree; results from flag-disabled arms were rejected.

4. The arc

Ten numbered studies and one registered bakeoff. The table is the record.

#	Added	Outcome	Terminal diagnosis
001	Recency plus similarity retrieval	PARTIAL, 2 of 3 bars	Similarity fired once in 32 turns. Thirty topics for 32 episodes compressed nothing
002	Consolidation, rule pinning, 120 turns	PARTIAL, 3 of 4	Similarity recovered buried facts; consolidation produced 52 topics
003	Long-term write path, four promotion filters	PARTIAL, 2 of 3	The weighted route was arithmetically unreachable — novelty and association were complementary values from one centroid, capped below threshold — so every promotion used the bypass, making it a novelty-spike detector
004	Long-term read path and arbitration	PARTIAL, 1 of 3	Retrieval ran on all 90 eligible turns with zero displacement; the store lacked the later-domain facts
005	Permissive capture, extractive distillation	PARTIAL	Absolute entity and number counts selected the longest responses. The salience metric was a verbosity detector; 2 of 4 domains formed
006	Length-normalized span selection	PARTIAL, 1 of 3	Formation reached 4 of 4 domains; records shrank about 28×, and count-based retrieval budgets silently broke
007	Character-budgeted retrieval	PARTIAL, 2 of 3	Best of the series. The model used all 10 delivered facts and invented none; 7 required facts were absent from the store
008	Rendering-by-floor factorial	STOPPED AT PRE-RUN GATES	No fill cap from 1 to 50 passed breadth and targeted gates jointly
009	Pure-STM null test	PARTIAL, null decisive	Same seed: 9.0 without the memory tier, 12.0 with it
010	1,000-turn endurance	STOPPED AT G2	Post-stop arms were budget-noncompliant by 67.9% and 68.2%; scores unaudited; one bar not evaluable

#	Added	Outcome	Terminal diagnosis
—	Retrieval bakeoff	MIXED	Query-time selection did not recover what formation missed

Two results from that table carry the argument forward.

Write-time selection cannot anticipate a query. Studies 003 through 007 are five attempts to decide, at write time, what deserves remembering. Each optimized a proxy satisfiable without the property it certified. The terminal diagnosis is specific: density, the best write-time salience signal, ranks the six hardest planted facts between 89th and 316th, and word-level inverse document frequency ranks them worse. These are facts like *photophores* and *ultramarine glaze* — rare technical phrases whose component words are common.

Moving selection to query time did not recover it. The bakeoff registered that repair as its central premise and refuted it. The best registered 32,000-character retrieval block surfaced **8 of 17** target facts, below the 11 of 17 the formation era reached. All 17 were present in the raw store; retrieval did not find them. The bakeoff also advanced none of three further pillars: graph retrieval cleared no advancement gate at any of eight edge types and three depths; query routing had an *oracle* upper bound of 6.09% against a registered 10% build threshold; approximate search degraded recall at synthetic scale.

Scores above are post-audit corrected values (§7.1). They are not a controlled series — runtime and response budgets changed across it — and the only clean architectural comparison the program has is Study 009’s same-seed pair.

Each of these is an unremarkable negative result alone. §5 is what they have in common.

5. The decomposition

5.1 The target was present and affordable

What the counts below measure. 6 of 17, 12 of 17, 14 of 17, 15 of 17 are *availability*: whether an item’s text was present in the block delivered to the model. They are not answer correctness. No arm of this selection work was ever scored end-to-end. §5.1.1 records a specific way the two come apart here.

Before asking why selection failed, the program asked what success would have cost. AR-001 computed, on the same store and under exact serialized accounting, the cheapest set of episodes making the breadth items available.

All 17 items are present; 76 of the 119 eligible episodes carry at least one. The exact minimum for 14 of 17 is **5,058 characters across five episodes**, leaving 26,942 characters of the 32,000-character budget unused. A greedy variant reaches 15 of 17 for 5,455. Even 17 of 17 costs 7,592. The most expensive domain, art, needs 3,182 — under a tenth of the budget.

Deployed selection made **6 of 17 available while spending 31,946 characters**.

Figure 1. The budget was never tight. Selection spent it on episodes carrying nothing.

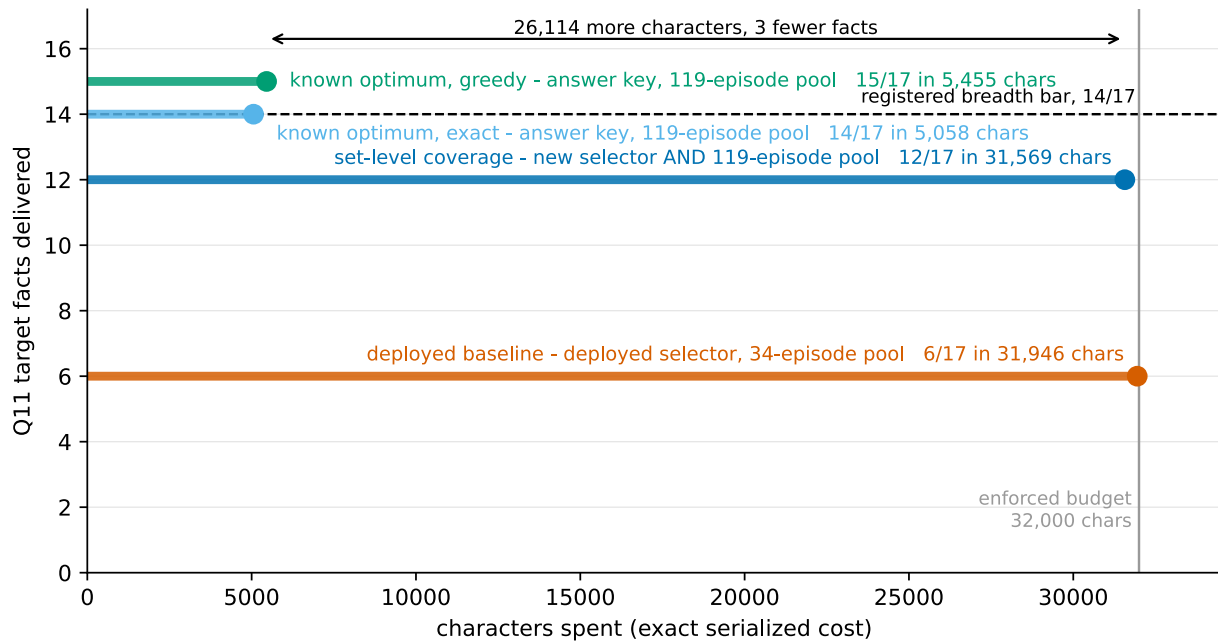


Figure 1. The budget efficiency gap. *The constraint is not capacity: the tallest result is also the narrowest.* Each horizontal stem runs from zero to the characters spent, at a height equal to the facts made available, over the same store at the same enforced 32,000-character budget. The deployed baseline reaches 6 of 17 for 31,946 characters; the shipped set-level configuration 12 of 17 for 31,569; the exact known optimum 14 of 17 for 5,058, leaving 26,942 unused; its greedy variant 15 of 17 for 5,455. **The top two bars change two things against the baseline, not one — the selector and the candidate pool.** §5.3 separates them: on the deployed 34-episode pool the same set-level configuration scores 5 of 17, below the baseline’s 6. Read this figure as the budget argument only. Both optima are computed with the answer key and are bounds, not methods. Sources: a0_baseline.json 7645e4746715a965, e005_results.json 07b714389697c6e5, achievability.json 770792d09e07978d.

Two cautions attach to the optimum wherever it appears below. It is computed with the answer key, so it is a bound and not a method. And there are two sets, easily conflated: the exact 14-fact optimum over turns {90, 112, 113, 115, 118} and the greedy 15-fact set over turns {90, 112, 113, 116, 118}. Everything the program calls “the oracle”, including every overlap figure here, is the greedy 15-fact set.

5.1.1 The optimum is mostly the conversation’s own earlier answers

Four of its five episodes are prior probe exchanges; only turn 90 is an original plant source. So “the target was reachable in 5,058 characters” partly means the facts had already been restated compactly in earlier answers, and retrieving those restatements is cheap. Achievability holds under the registered eligibility rule — any episode before the probe turn counts — and not under a stricter plant-source-only rule.

There is a sharper consequence, and it bounds how much any count in §5 means. A selector preferring prior answers propagates prior errors, **and this probe’s earlier answers were largely wrong.** Availability is scored on an item’s presence in the delivered text, so an earlier answer naming the right entity inside an otherwise incorrect response makes that item “available”.

Part of the 15-of-17 optimum, and of any configuration scoring well by recovering those four episodes, therefore consists of delivering the conversation’s own earlier mistakes with the correct nouns in them. The decomposition survives this — it concerns which episodes get selected, not whether they are true — but “makes 15 of 17 available” and “would help the model answer correctly” are further apart than an availability count suggests, and nothing in this program measured the distance.

5.2 The candidate pool binds first

Selection runs over whatever the pre-filter admits. DR-002 froze the shipped configuration and varied only pool membership — same store, same renderer, same embedding.

Pool	Candidates	Facts	Domains	Known-optimum overlap
deployed pre-filter	34	5/17	2/4	1/5
cosine top-100	100	9/17	3/4	0/5
full eligible store	119	12/17	4/4	4/5

Widening the pool moves the same configuration from 5 of 17 across 2 domains to 12 across 4. Figure 2.

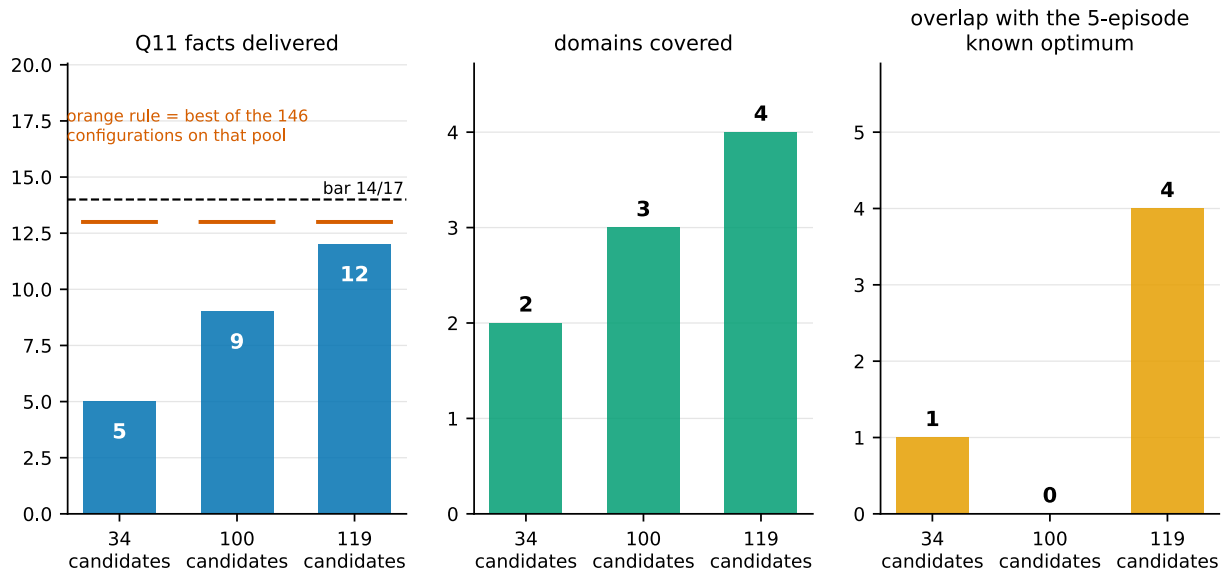


Figure 2. Pool ablation. Widening the candidate pool from 34 to 119 episodes, with the selector frozen, moves the same configuration from 5 of 17 across 2 domains to 12 of 17 across 4. Facts, domains, and known-optimum overlap at three pool sizes, everything except pool membership held fixed. Dropping only the 19 lowest-cosine episodes to form the 100-pool costs three facts, the whole art domain, and all optimum overlap, though four of the five optimum episodes survive the cut — the selector clusters over the pool, so tail removal reshuffles the objective rather than removing options. The orange rule marks the best of 146 configurations on each pool: 13 of 17 on all three, which is why the pool’s binding effect must be read in domain coverage. Sources: configuration_sweep.csv 1ad625d10fb988f9, pool_secondaries.csv 5987d05846c64f97.

Two things keep this honest. It is a **frozen-configuration** readout, not a sweep: the best of 146 configurations reaches 13 of 17 on all three pools, so the pool’s binding effect is properly read in domain coverage rather than in that maximum. And on the deployed 34-episode pool **no configuration covers four domains at all**, because the art domain has no representative anywhere in the top 34. That is a statement about what the pre-filter makes possible, not about how well any selector searches.

The middle row deserves a second look. Dropping only the 19 lowest-cosine episodes to form the 100-pool costs three facts, the whole art domain, and *all* overlap with the known optimum — though four of the five optimum episodes survive the cut. The selector clusters over the pool, so removing the tail reshuffles the objective rather than merely removing options. Pool size does not predict what removal costs, which is why §6.4 rejects pool trimming as a cost control.

5.2.1 Why the pool binds: the ordering is inverted at the top

DR-002 registered its rule before computing any rank: *any fact-bearing selection at cosine rank 80 or worse means cosine ordering is the wrong prior for breadth*.

It fired. The worst fact-bearing selection sits at rank 86 and carries two of the four art items. Around it:

- **The four highest-cosine episodes in the store carry zero target facts.**
- The first fact-bearing episode is at rank 5; the last still-needed item does not appear until rank 87 of 119.
- Both art contributors sit at ranks 50 and 86 — which is exactly why the deployed 34-episode pool cannot reach four domains.
- The five known-optimum episodes sit at ranks 14, 20, 22, 86, and 112. The shipped configuration recovers four, including the one at rank 86, and misses only the deepest.

Figure 3. The failure is not that the ordering is noisy. At the top it is anti-correlated: the most query-similar episodes are the least informative for this probe, and one of the two domains deciding the gate is unreachable before rank 50. That a set-level objective reaches rank 86 at all bounds the claim — the objective partially compensates for a bad ordering, and does not compensate fully.

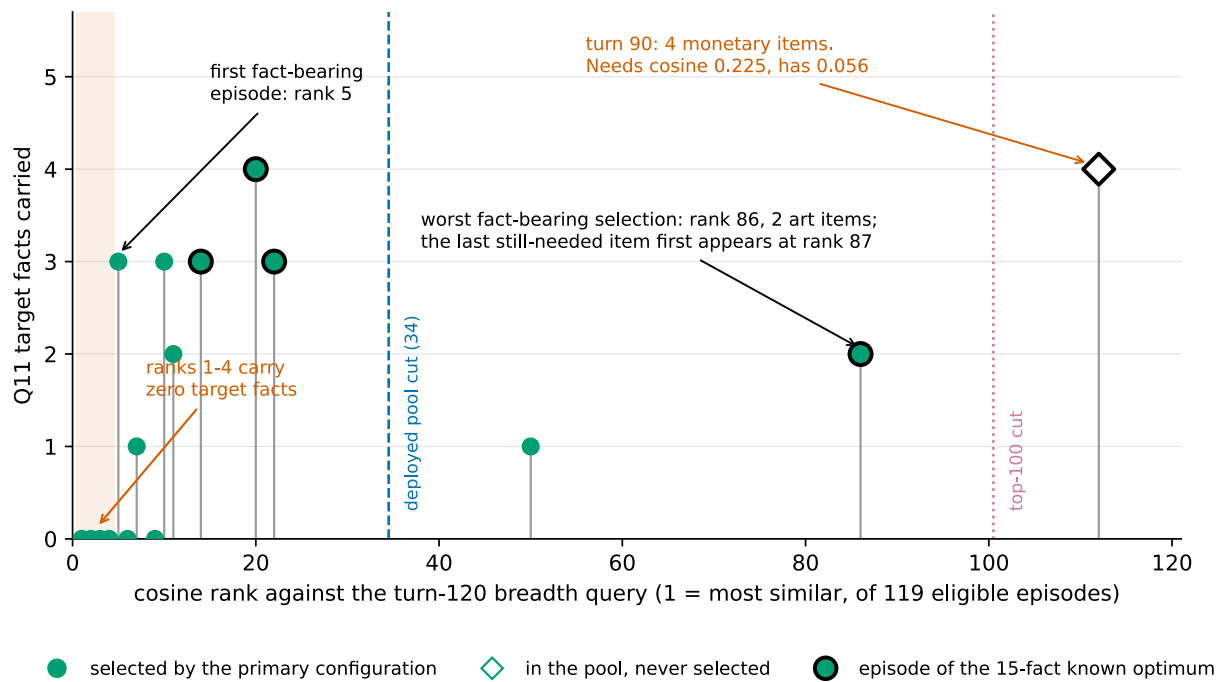


Figure 3. Cosine rank against fact content. *On this corpus, the enumeration probe’s target facts sit outside the top of the cosine ranking, and the deployed pool cut removes an entire domain.* Horizontal axis: cosine rank against the turn-120 breadth query over the 119 eligible episodes. Vertical axis: target facts carried. The four highest-ranked episodes carry zero; the first fact-bearing episode is at rank 5 and the last still-needed item does not appear until rank 87. The five episodes of the 15-fact known optimum are marked at ranks 14, 20, 22, 86 and 112. Both art contributors lie at ranks 50 and 86, so the deployed 34-episode pool contains no art episode and cannot reach four domains at any setting; the 100-episode pool excludes the rank-112 episode carrying four monetary items. Only the 16 episodes whose ranks are committed are plotted — the 15 selected plus the rank-112 miss; ranks for the other 103 were never committed (§8.8). The rank-4, rank-5 and rank-87 readings are committed structural values drawn as annotations, not inferred from the plotted points. Sources: selection_ranks.csv 6fdff4022997ab83, cost_comparison.csv 1ca40da99315c719, generality_batched.json 7e1fa13ef71a8077 rank 20 supersedes the published 21 per ERRATA.md, 2026-08-01.

5.3 The objective binds second, and only after the pool

The deployed rule ranks each episode against the query independently and takes the best until the budget fills. That rule cannot represent redundancy: an episode’s value does not depend on what is already selected. E005 replaced it with three set-level objectives — maximal marginal relevance, facility location, and relevance plus cluster diversity — swept over 146 configurations per pool at the enforced budget, with zero inference calls and a byte-identical rerun. The best gate-passing configuration makes **12 of 17 items available across 4 of 4 domains at 31,569 characters**, preserving all 16 targeted items and recovering 4 of the 5 known-optimum episodes.

That headline moves two variables at once. The deployed baseline is the deployed selector on the deployed 34-episode pre-filter; the 12 of 17 is a set-level selector on the full 119-episode store. Selector and pool both changed, so it is not a selector result and this paper does not use it as one. The per-pool minima across the same 146 configurations make the confound concrete:

Pool	Worst of 146	Best of 146	Frozen shipped config	Deployed baseline
deployed pre-filter, 34	4/17	13/17	5/17	6/17
cosine top-100	5/17	13/17	9/17	not applicable
full eligible store, 119	7/17	13/17	12/17	not applicable

The baseline column has one entry because the baseline has no pool variable: it is the deployed pipeline’s pre-filter and selector together, never run against a wider pool. The blanks are not zero measurements.

Read the first row. On the pool the system actually used, set-level configurations fall as low as 4 of 17, and **the configuration this program ships scores 5 of 17 there — below the 6 of 17 baseline it replaces**. The selector is not a free improvement. It requires the wider pool, and without it is a regression.

The honest within-pool statement is the third column against the fourth: holding the deployed pool fixed, the best of 146 set-level configurations makes 13 of 17 available against the baseline’s 6. That is a selector effect, on one pool.

Three further results matter more than the headline.

The highest-scoring selector was the worst one. Facility location reached 13 of 17, the highest raw count, and passed no gate at any setting, because it delivered the monetary domain 0 of 4 every time. It improved the total by abandoning a domain. Only the per-domain check caught it; a reader looking at fact counts would have shipped it. Figure 4.

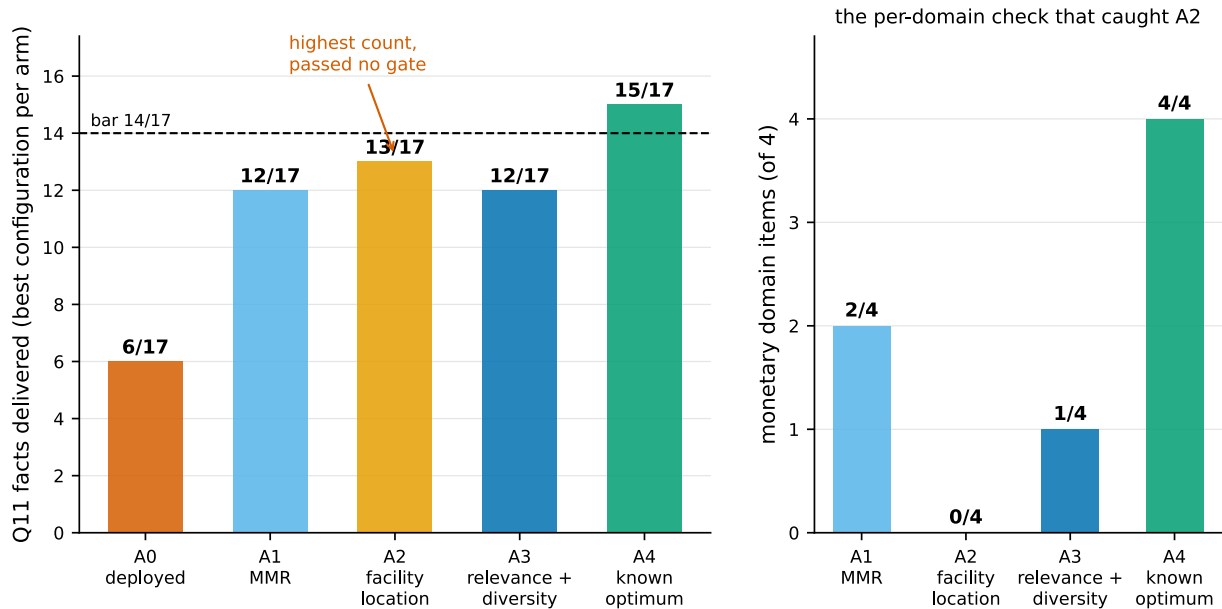


Figure 4. Selector comparison. *The selector with the highest raw fact count delivered nothing from one domain and passed no gate.* Best configuration per arm on the 119-candidate pool at the enforced budget, with the monetary domain broken out. Facility location leads on count at 13 of 17 and delivers monetary 0 of 4 at every setting; the shipped relevance-plus-diversity configuration reaches 12 of 17 across all four domains. On this pool all 146 configurations beat the deployed 6 of 17; on the deployed 34-episode pool they fall as low as 4 of 17 and the shipped configuration scores 5 of 17 (\$5.3). 137 configurations preserve 16 of 16 targeted items, so targeted recall does not separate the arms — itself the finding. Sources: configuration_sweep.csv 1ad625d10fb988f9, a0_baseline.json 7645e4746715a965, e005_results.json 07b714389697c6e5, achievability.json 770792d09e07978d.

A parameter registered as inert was not. Cost scaling was predicted to make no difference on a budget with slack. The budget is slack for the optimum and not for a selector registered to fill it, and the prediction failed.

The search is not the limit. The bound here is data-dependent, not a worst-case constant: at the final greedy set, each unselected candidate’s marginal gain per unit cost is computed, the remaining budget filled fractionally in that order, and the result added to the achieved objective value to bound the optimum. On the primary 119-episode pool, greedy sits between **0.9548** and **0.9996** of it across the 135 configurations where the bound is computable; the shipped configuration is at 0.9927. A better search over the same objective has almost nothing left to find.

Two scope notes, because this number is easy to quote wrongly. Widening the population to all three pools — 405 computable configurations — moves the minimum to 0.9536; the program’s ledger reports 0.955, which is the primary-pool minimum to three places and is the figure this paper uses. Neither value supersedes the other and no published number changes; they count different configurations. And the bound exists only for the two arms with a set function. All 33 non-computable rows are the maximal-marginal-relevance arm, so this reading covers facility location and relevance-plus-diversity and says nothing about MMR’s search quality.

5.4 The similarity floor binds last

After the pool is widened and the objective replaced, one known-optimum episode remains unselected: turn 90, carrying four monetary items — the reason monetary is the shipped configuration’s weakest domain at 1 of 4.

DX-001 ran a replay gate first, reproducing 146 of 146 committed payload hashes byte-for-byte before reporting anything. The gate earned its place; §7.4 records what it caught.

No configuration in the registered space selects turn 90. That count is weaker evidence than it looks and the paper does not lean on it: the 146 configurations are a grid over three parameters, so sweeping the diversity weight eleven times is closer to one chance repeated than to eleven chances. The arithmetic is the argument.

The registered prediction was cluster collision — the episode shares a cluster with a selected one, so the diversity term goes unpaid. That prediction is **wrong**, refuted twice over: the episode's cluster is never entered by any selection, so the diversity term was payable in full at all 15 steps, and a counterfactual paying it in full wins at no step.

What remains is arithmetic. To win at its best step the episode needed a query relevance of **0.225**. It has **0.056**. Only 20 of the 119 episodes clear that bar, so it would have to be a different episode by cosine, not a better-weighted one. Across 132 walks of the parameter space its best rank anywhere is 4, never 1.

This is a floor. No reweighting of an objective built on that similarity reaches it, which is why the registered no-change branch fired and 12 of 17 ships with the miss characterized rather than tuned away.

5.5 What the inversion does not explain

§5.2.1 invites a larger claim — that every mechanism in this program, and much published architecture in this space, ran downstream of a broken candidate ordering. DR-002 tested that reading and **measured it false**.

On the eight targeted probes, cosine ordering places every needed item **inside rank 2**, and the top four candidates carry a target item on every one. That is near-optimal, and it is why targeted recall runs at 60 of 60 and why 137 of the 146 configurations preserve 16 of 16 targeted items without effort.

The inversion also fails to explain most of §4. Not the formation-side failures, which ran at write time upstream of any retrieval filter. Not Study 003's arithmetically unreachable promotion route. Not Study 007, where the model used all 10 delivered facts and seven required facts were simply absent from the store. It is contradicted by the bakeoff's routing oracle, which assumed perfect selection and still ceilinged at 6.09%, and by widened raw short-term memory, which delivered all six formation-blind facts — the ones density and inverse document frequency both missed — with no selection filter at all, the model using five correctly.

One limit decides how much §5.2.1 can carry: the comparison is **eight probes against one**. The program has exactly one enumeration question, so the entire enumeration side — the top four carrying zero, the rank-87 reading, the registered rule firing — is a single instance, and one instance cannot establish a query type.

So the claim is what was measured: **on this corpus, one enumeration probe behaves completely unlike the eight lookup probes, and the mechanisms this program built for breadth all ran downstream of the ordering that probe exposes**. It unifies the breadth failures. It does not unify the program, and it does not yet describe a category of query.

5.5.1 The cheapest test of whether this is a corpus artifact

The program's own description of its hardest facts is that they are rare technical phrases whose component words are common. That is a *lexical* property, and it predicts the observed effect directly: an embedding will place such a span far from a query phrased in ordinary words. If that is the mechanism, §5.2.1 is a finding about this corpus's planted vocabulary and generalizes only to corpora whose target facts are similarly distinctive.

The discriminating measurement is cheap and was not run: correlate each fact-bearing episode's cosine rank against the lexical rarity of its key phrases, on the committed store. The ranks and the phrases are both already committed, and the program has rarity scores from an earlier audit. **If rank tracks rarity, the inversion is vocabulary. If it does not, the corpus objection weakens considerably**. This is the single measurement that would most change how much weight §5.2.1 deserves.

5.6 The three constraints, and why the order is forced

Constraint	Binds on	Bound, on this corpus
Candidate pool	domain coverage, and part of the fact gap	The art domain has no representative in the deployed 34-episode pool, so no rule of any kind reaches four domains there. With the selector frozen, $34 \rightarrow 119$ candidates moves 5/17 across 2 domains to 12/17 across 4
Selection objective	the remaining recoverable facts	Holding the deployed pool fixed, the best set-level configuration reaches 13/17 against the baseline’s 6/17. Greedy runs at 0.9548–0.9996 of a data-dependent bound on the primary pool, so the objective and not the search is the limit
Similarity floor	the irreducible residual	0.056 against the 0.225 required, a shortfall of 0.169; only 20 of 119 episodes clear the bar; unreachable by any reweighting of this objective

The order is forced, and the argument for it is structural rather than numerical. **The art domain has no representative anywhere in the deployed 34-episode pool.** Its two contributors sit at cosine ranks 50 and 86 (§5.2.1), so no selection rule of any kind — set-level, per-item, or hand-written — can deliver an art item from that pool. Four-domain coverage is not merely hard there; it is unavailable. Objective work cannot recover a domain the pre-filter never admitted.

The pool has to be widened first. That follows from what the pool contains, not from any measured comparison, and nothing about a single run or a single seed bears on it.

A measured result points the same way and is worth stating for its vividness, with its fragility attached: run on the deployed pool, the configuration this program ships scores 5 of 17 against the baseline’s 6 of 17 — the objective fix, applied alone, is a small regression. That is **one count, from one unreplicated run**, and a reader is entitled to weigh it lightly. Note also that the best of 146 configurations reaches 13 of 17 on that same pool, so the shipped cell is not the only cell one could point at.

Neither observation is load-bearing. **The structural claim carries the ordering by itself**, and the 5-against-6 is illustration.

That is the operational content of the decomposition and what a practitioner should take from §5.

One clarification the title invites. “Selection, not capacity” contrasts with the *character budget*, which was never binding: the target cost 5,058 of 32,000 characters while deployed selection spent 31,946 to deliver a third of it. The candidate pool is a capacity limit of a different kind — on how many episodes the selector may consider — and it binds first. Both hold. What the program spent four studies believing, and what is false, is that the character budget was the constraint.

We have not found this decomposition reported elsewhere in this space. The reason is likely mechanical: computing it requires a per-fact known optimum on the same store, which requires both an answer key and exact-cost accounting, and evaluations reporting end-to-end scores against a benchmark do not usually carry either.

6. What survives

None of this section was run live. The 12 of 17 result is offline availability, registered outcome PROMOTION_ELIGIBLE, meaning it may be promoted to a live study and has not been. No inference run of the shipped configuration exists, so nothing here reports what a model scored with it. The component-level guarantees below are tested; the retrieval result they carry is not.

6.1 What was removed

Each mechanism below was built, measured, and closed by the result beside it. They do not correspond one-to-one with the studies: several fell to the same study, and some outlived the study that first weakened them.

Removed	Killed by
Distillation and “dreaming”	Five studies; query-blind selection cannot anticipate a later query
Promotion filters	The weighted route was arithmetically unreachable; every promotion came via bypass
Topic layer and consolidation	52 topics for one 120-turn conversation; 12 domains collapsed to 2 at 1,000 turns

Removed	Killed by
Associative graph from co-activation	No configuration cleared its advancement gate
Query-type routing	Oracle ceiling 6.09% against a 10% threshold
Approximate nearest-neighbour search	Recall degraded at synthetic scale
Query segmentation	Improved its matched-budget baseline from 6/17 to 10/17 and still failed its locked 14/17 bar
Attention-derived term selection	Run as an oracle over 714 candidate cue rows: 0 reached the retrieval threshold
Entity extraction as primary index	Zero entities in the target span
Density and inverse document frequency for formation	Rank the six hardest facts 89th–316th, and worse
Rule detection and persistence	Failed at 1,000-turn scale

6.2 What remains

An append-only verbatim store. A recency window. Cosine-threshold similarity retrieval for targeted queries. A set-level coverage objective for selection. Everything packed at exact serialized cost against one budget.

There are no generative model calls anywhere in the memory path. Nothing in it asks a model to write text about the store. That is the property Study 005 established and the one every removal preserved. It is not the absence of model calls. `context()` embeds the query on every call, and `append()` embeds every episode, so an embedding model must be resident. What follows from the architecture is that the memory path emits no generated text, and that its output is reproducible **given a pinned embedder** — which is why the library asserts a sentinel vector hash on every store open rather than assuming one (§7.4).

That is the whole architecture, and it is smaller than what this program started building. It contains none of the mechanisms listed above. We make no claim about how it compares to systems that were never run here.

6.3 Why a practitioner should care

Interpretation, not measurement. The properties are measured; the causal story about why they hold is a reading of this program’s history.

The surviving design is reproducible and provenance-preserving, and both track a negative result rather than a design goal. It is reproducible because distillation was removed, and distillation was the component whose output could vary run to run: `context()` is a pure function of store state, query, and budget, verified byte-identical across two processes. It preserves provenance because every delivered character is a stored episode verbatim — there is no generated text about the store to be wrong, which follows from removing the mechanisms that generated such text.

Stated plainly, as a reading rather than a result: **the properties that make this component deployable were bought by the failures, not by the design.** A program that had succeeded at distillation would have shipped something harder to test and harder to audit.

The extraction is certified rather than assumed: all 132 committed selection records and all three committed rendered blocks reproduce their SHA-256 byte-for-byte through the library, with the full suite at 1,007 tests.

6.4 What it costs

These results come from the 1,000-turn endurance run, not the 121-turn corpus §5 uses. Nothing here establishes a §5 result at 1,000 turns, and nothing in §5 establishes these at 121.

Three things could grow as a conversation lengthens. Only one binds.

Delivered context is bounded, because it is enforced. Replaying the 1,000 committed episodes through the library at a 32,000-character budget, the delivered block breaches the budget on 0 of 1,000 turns and its 95th percentile moves +18 characters across the final five 100-turn buckets. The same block also **truncates on 895 of those turns**, dropping up to 70 episodes and wanting up to 65,864 characters. It is bounded because a ceiling binds during selection, not because demand is small. Both readings belong together; the first alone would be the kind of surrogate this program keeps catching.

Disk is cheap. 4,743 bytes per turn at the margin, 86% of it embeddings. About 48 MB at 10,000 turns. Nothing there ends continuous operation.

Latency binds. 190 ms at 1,000 candidates, clustering 81% of it and rising. The stated horizon is comfortable to a few thousand episodes and unusable in an interactive loop somewhere before 10,000. Figure 5, right panel.

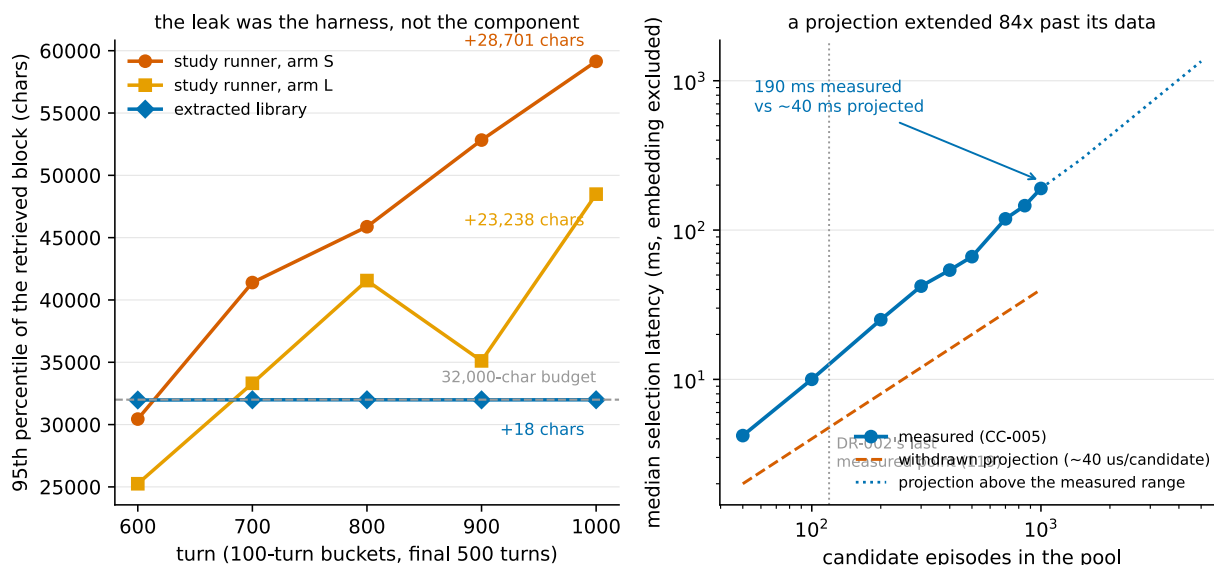


Figure 5. Growth and cost. *Growth belonged to the harness; cost was five times the projection.* Left: the 95th percentile of the retrieved block per 100-turn bucket over the final 500 turns of the 1,000-turn run. In the study runner the block rises 23,238 characters in one arm and 28,701 in the other, still setting records in the last bucket; replayed through the extracted library at the same budget it moves +18 characters and breaches the budget on 0 of 1,000 turns — while truncating on 895 of them, so it is bounded because enforced, not because demand is small. Right: measured median selection latency against candidate count with the withdrawn linear projection overlaid; 190 ms measured at 1,000 candidates against about 40 ms projected, exponent 1.25 over 50–1,000 where the earlier sweep found 0.96 over 20–119, and clustering’s share rising from 37% to 81%. Values above 1,000 candidates are projections, drawn dashed. Sources: dx002_results.json f8ab79ab041cb3e3, ge0_growth_gate.json 350fe20bbc3beed9, latency_curve.csv 0d2b8075ff5a971f, growth_measurement.json 20d65a018e28b03f.

Those milliseconds are one machine’s. The runtime is llama.cpp with a 27B generation model at UD-Q6_K_XL, one slot, fixed seed, speculative decoding disabled, and Qwen3-Embedding-0.6B over SQLite with sqlite-vec. Selection timings exclude embedding entirely — query and episode vectors are already resident — and are medians over repeated runs on a single machine. Only the exponent and the clustering share plausibly transfer.

The obvious fix — keep the pool small by dropping low-similarity episodes — is the one operation this program measured to break retrieval (§5.2). So retention is unbounded by policy, the trimming knob carries an `unsafe_prefix` and the finding in its docstring, and the horizon is stated rather than engineered around.

7. Self-audit and corrections

Everything in §5 rests on this program’s own measurements, scored by its own raters, against its own rubric. The reason to extend it credit is that the program audited itself and published what it found. All six items were caught by gates the program wrote, and all six are in `ERRATA.md`.

7.1 Four corrections, in brief

The scoring audit removed the program’s only success. A blind re-scoring of 222 committed items across Studies 001–009 changed 19. Study 002’s iterative arm fell 13.0 → 8.5, because a truncated reasoning block had been credited as a complete response; Study 001 lost the program’s only `VALIDATED` verdict. The residual estimate is extrapolated, and its precision is reported with it: 3 disagreements in a 26-item control sample projected across 143 unreviewed items gives 16.5 expected errors, but the 95% Clopper-Pearson interval on 3 of 26 runs from 2.4% to 30.2%, admitting **anywhere from about 3 to about 43**. A paper whose §7.3 is about an interval being misread should not present one number where the data supports a range that wide.

Every study on record ran over its stated budget. Study 010 reported two retrieval blocks at 31,991 and 31,847 characters and called them near-saturation of a 32,000 budget. Replayed character-for-character, their serialized lengths were 53,726 and 53,839 — **67.9% and 68.2% over**. The old accounting

counted source text and omitted per-episode tags, metadata, and separators. Scores did not change, but they describe a budget-noncompliant arm, and the compact-store conclusion built on the undercharged figures was withdrawn.

A validator invalidated a published curve. A probe-order check verifies mechanically that every rubric-required fact is planted strictly before the probe asking for it. It found degradation probes requesting facts not yet planted, invalidating a published curve. The check now blocks artifact lock.

A projection ran 84× past its data. A pre-registered budget quoted about 40 microseconds per candidate at exponent 0.96 and projected 40 ms at 1,000 candidates. The source sweep covered **20 to 119**. Measured at 1,000, the cost is **190 ms, about five times the projection**, at exponent 1.25. The correction is to the range attributed and the extrapolation built on it, not to the original measurement. Figure 5, right panel.

7.2 The same query text returns different vectors

DX-001's replay gate failed on its first attempt. The cause was not the query text but the shape of the embedding call: E005 embedded nine probe queries in one batch, the replay embedded one alone. The two vectors agree to a cosine of **0.999837**, with a largest single-component difference of **0.217**, and that difference flips **6 of 146** committed selection payloads.

A 0.999837 agreement reads as identical and is not. Reproducing a retrieval result requires reproducing the call shape, not only the text. The shipped library now embeds a fixed sentinel under the pinned call shape on every store open and asserts its hash against the one recorded at first open; drift raises rather than warns.

This has one direct consequence for reproduction. The 12-of-17 result was produced with the breadth query embedded in a nine-query batch; the shipped `context()` embeds a single query alone. The primary configuration is **not** among the six payloads that difference flips, so the headline reproduces under either — a checked fact rather than a safe assumption, and six other configurations do not have it.

It is also evidence about embedder sensitivity generally, which §8.4 collects.

7.3 A diagnostic written to catch surrogate failures nearly committed one

This is the most instructive of the six.

The recurring failure class this program tracks is a check that can pass while the property it certifies is false. DX-002 was written to determine whether a 1,000-turn run's context was still growing. Its decision rule was a three-clause conjunction; its implementation checked one clause — whether the terminal slope's 95% confidence interval contained zero.

It did, for every component of the prompt, in both arms. The diagnostic returned “bounded”.

The third clause was *no unbudgeted component climbing*, and one was. A block whose 95th percentile rose from 25,253 to **48,491 characters** across the final five buckets, still setting records in the last bucket, was reported as flat. The interval was wide because the series are sawtooths with autocorrelated residuals; it was measuring statistical power and was read as evidence of boundedness. The smallest slope the data could distinguish from zero was about 17 characters per turn — 17,203 characters of drift over 1,000 turns the fit would not have caught.

The rule was replaced with two readings assuming nothing about noise: whether the final bucket still holds the maximum, and how the terminal window compares against the one before it. The verdict flipped, and the near miss was written into the decision record rather than quietly repaired.

The finding that followed is Figure 5: the leak belonged to the study runner, which carried the recency window and the retrieval tier on separate budgets, not to the extracted component, which routes both through one.

7.4 What the gates missed

Every error above was caught by a gate this program wrote, which invites the obvious question and deserves the honest answer.

Between about 3 and about 43 scoring errors are estimated to remain unreviewed, point estimate 16.5. Study 010 was outside the scoring audit entirely, so its exploratory scores are not comparable to the corrected series. The mechanism seal for one tier was computed over mixed line-ending representations and referenced a database file never committed. And runtime independence was never measured: every number here comes

from one model at one quantization on one machine, with §7.2 giving positive reason to expect a different embedder would move the §5 results.

8. Limitations

Each item names what would settle it. §5 states its own scope where it matters; this section is the complete list rather than a restatement.

8.1 One corpus, one seed, no variance, no calibration. Every comparison is a single run at a fixed seed. There is no error bar anywhere and no meaningful significance test. Where this paper reports a difference — 6 of 17 against 12 of 17, or the load-bearing 5 against 6 of §5.6 — it is one measurement against another. Study 003 retired external baselines and nothing restored them; LoCoMo and LongMemEval were adopted in principle and never run, so nothing here establishes where this program sits relative to published systems. Boundedness claims are statements about a 1,000-turn horizon and say nothing about 10,000. *Settled by:* repeated runs at multiple seeds, and running one external benchmark.

8.2 Breadth rests on a single probe. The program has exactly one enumeration question. Every breadth number here — 6 of 17, 12 of 17, 14 of 17, the rank-87 reading, all of §5.2.1 — comes from it. A single probe cannot support a claim about enumeration in general, and this paper does not make one. *Settled by:* more enumeration probes across more domains.

8.3 AI raters, AI adjudicators. Scoring used three blind passes with registered adjudication triggers, but the adjudicators were subagents, not humans. The control sample disagreed at 11.54%. *Settled by:* human adjudication of the same items.

8.4 One runtime, and positive evidence of fragility. One model, one quantization, one machine, one embedder. Whether §5's results survive a *different* embedder is unmeasured — but this is not a blank absence of evidence, and it would be convenient to call it one. §7.2 shows the *same* embedder, given the same text under a different call shape, returning a vector agreeing to cosine 0.999837 that flips 6 of 146 committed payloads. A perturbation far smaller than a model change already moves 4% of results, so the reasonable prior is that §5's specific numbers are embedder-dependent. *Settled by:* rerunning the E005 sweep under a second embedder.

8.5 Planted facts may not represent natural conversation. The corpus is constructed, and §5.5.1 gives a specific reason to suspect the inversion is a property of the planted vocabulary rather than of retrieval, along with the cheap measurement that would discriminate. It was not run.

8.6 Availability is not correctness, and the known optimum is mostly prior answers. §5.1.1 in full: four of five optimum episodes are prior probe exchanges, this probe's earlier answers were largely wrong, and an item counts as available if its text appears — however wrong the surrounding response.

This is the paper's largest structural weakness, and unlike the others it has a defined remedy rather than an aspiration. *Settled by:* **LV-001**, a pre-registered two-arm live run of the shipping configuration against the deployed baseline at one seed ([experiments/components/live_validation/LV_001_pre_registration.md](#)). Its thresholds and both reporting outcomes are fixed in advance, including what this paper must change if the offline advantage fails to convert. The design is committed; the run needs a runtime this repository does not carry, and has not been authorized.

8.7 Amendments exist after results. Twelve in the bakeoff alone. The program records per amendment whether it preceded the result it affects, and applies a legitimacy test permitting corrections to measurement units and protocol contradictions while forbidding making a criterion easier once results are known. The record is published so a reader can disagree with individual calls.

8.8 Figure 3 is drawn at the resolution the artifacts support. Per-episode cosine ranks were committed for 16 of the 119 candidates; the rest were never committed, and recomputing them requires the carried embedder under the batched call shape of §7.2, which is not in the repository. The structural readings the figure annotates are committed values, but the full curve is not drawn because it cannot be drawn honestly.

9. Conclusion

Eleven pre-registered efforts on one program produced one architecture worth keeping and a measurable account of why the rest did not work.

For a practitioner, the useful part is the subtraction. Every mechanism the program built failed its own gate; what is left is an append-only verbatim store, a recency window, similarity retrieval, and a set-level coverage objective, with no generative model calls in the memory path. That component is reproducible given a pinned embedder and auditable line by line, and §6.3 argues both properties followed from the removals rather than from foresight. If a memory component in your system makes generative calls, this program’s experience is that they bought less than they cost — on one corpus, with no live comparison run. For a researcher, the useful part is the decomposition and the order inside it. Retrieval failure here was not one thing. The candidate pool decided what could be seen, the objective decided what was worth taking, and a similarity floor decided what was unreachable at any weighting. The order is forced rather than tidy, and structurally so: one of the four domains has no representative anywhere in the deployed shortlist, so no objective can recover it from there. Widening the pool is not the first fix because it measured better. It is first because the alternative is impossible. Separating the three required a per-fact known optimum on the same store — a measurement costing an answer key and exact cost accounting, and buying a sharper question than an end-to-end score.

The observation most worth testing elsewhere is the narrow one, and it is a single instance. On this corpus, for the one enumeration probe this program has, the four highest-cosine episodes carried none of the target facts and the last needed item sat at rank 87 of 119 — while the same ordering placed every needed item inside rank 2 on all eight lookup probes. One probe cannot establish that enumeration queries are a category with different retrieval needs. It is enough to make the question worth asking on a corpus this program did not build, and §5.5.1 names the cheaper measurement that would first tell us whether the effect is about retrieval or about the vocabulary these facts were planted in.

The program’s own summary of eleven efforts is that the model used what it received. At the hardest probe it used all ten available facts and invented none. The failures were delivery failures, and delivery turned out to be a selection problem sitting on top of a candidate set already narrowed by the wrong rule.

Appendices

- **A. Claim-to-artifact table** — `paper/CLAIM_TO_ARTIFACT.md`. Every claim with its committed artifact and hash; two claims cut or demoted for want of one.
- **B. Study table** — §4 above; full reports under `experiments/study_NNN/`.
- **C. Amendment record** — `per-study/amendments/` directories, each with its before/after-result status.
- **D. Corrections index** — `ERRATA.md`, cross-referenced from §7.
- **E. Reproduction** — `paper/REPRODUCTION.md` and `paper/reproduce_headline.py`. Rebuilds the 5,058-character exact optimum from the committed turn log through the installed library and checks its length and SHA-256 against AR-001’s committed values. Verified in a clean virtual environment containing only `episodic`. The selection results cannot be reproduced this way, because the store’s vectors were never committed; the appendix says so.
- **F. Spec reconciliation** — `paper/notes/EVIDENCE_INDEX.md` §1, recording six places where the authoring specification and the committed artifacts disagreed and the artifact won.
- **G. Review record** — `paper/reviews/`: two adversarial cycles, a slop audit, and the three-reader readability review that prompted this restructure.

Typeset PDF. `paper/Selection_Not_Capacity.pdf`, built from this file by `scripts/build_paper_pdf.py` (pip install `typst`). The PDF has no independent source: it is generated from this Markdown, figures are placed at the paragraph that first cites them, and figure numbering comes from here rather than from the typesetter, so the numbers cannot drift from the prose. If the two ever disagree, this file is right and the build script is broken.